

A Software Architecture for the Dynamic Path Planning of an Autonomous Racecar at the Limits of Handling

Johannes Betz, Alexander Wischnewski, Alexander Heilmeyer, Felix Nobis, Leonhard Hermansdorfer, Tim Stahl, Thomas Herrmann, Markus Lienkamp

Chair of Automotive Technology
Technical University of Munich
Boltzmannstraße 15
85748 Garching, Germany
Corresponding Author: Johannes Betz, betz@ftm.mw.tum.de

Abstract—Based on a software architecture for autonomous driving presented and tested in an autonomous level-5 racecar in 2018 this paper describes in detail the evolutionary enhancement of this software architecture. The architecture combines the autonomous software layers perception, planning and control, which were modularized in the core software. The focus of this paper is the detailed description of how we enhanced the software with a module for an object list creation, a module for the behavioral planning and a module for the creation of dynamic trajectories. These enhancements allow the car to overtake other cars and static obstacles autonomously when driving on the race track. Furthermore, we present with a high novelty value the software module for a vehicle performance maximization, which consists of a control performance assessment and a friction estimation. The software architecture displayed in this paper will be tested and evaluated in the Roborace Season Alpha in 2019.

I. INTRODUCTION

Autonomous driving is seen as a game changer in many respects: a solution for the increasing volume of traffic due to urbanization and globalization, entry into new business areas in logistics and car sharing as well as for the reduction of road traffic accidents [1]. The development of autonomous vehicles is a complex process that must take social, economic, and above all, technical parameters into account. The supreme discipline here is the development of autonomous level-5 vehicles, in which no humans need to be present anymore. Furthermore, from a technical point of view the level-5 vehicle must cover all possible extreme situations of a normal vehicle with regard to speed, longitudinal and lateral accelerations as well as functionality of the algorithms. Today's passenger cars only show level 1-2 functions and the knowledge of their software architecture is limited. This is especially the case in terms of extensibility of the software to further functionality.

For these reasons, Roborace [2] wants to establish the first racing series for autonomous vehicles (figure 1) as a development platform for autonomous driving software. The Technical University of Munich (TUM) decided to participate in this racing series and developed an autonomous

driving software for level-5 vehicles [3], [4]. This software was first tested in a racing event on a Formula E track in Berlin in May 2018. Because of the great success of this software the TUM team decided to keep the software architecture and enhance its functionality. The new functions are aiming for a more dynamic and safe behavior of the vehicle. In addition it is the goal to integrate software functions that constantly adapt the vehicle to the current track conditions for reaching the handling limits.



Fig. 1. Devbot 2.0 for the Roborace Season Alpha 2019

II. RELATED WORK

On the first sight, defining an architecture for an autonomous driving software is a quite simple task. Pendelton et al. [5] define three core parts called perception, planning and control. Gaining information from the sensors (e.g. LIDAR, camera etc.), the perception part is creating a presentation of the current environment around the vehicle. In addition, the car has to localize itself either on a known map (LIDAR localization, HD-maps) or in unknown area (odometry, GPS). With this information the car can plan its route, behavior and the final trajectory that will be driven.

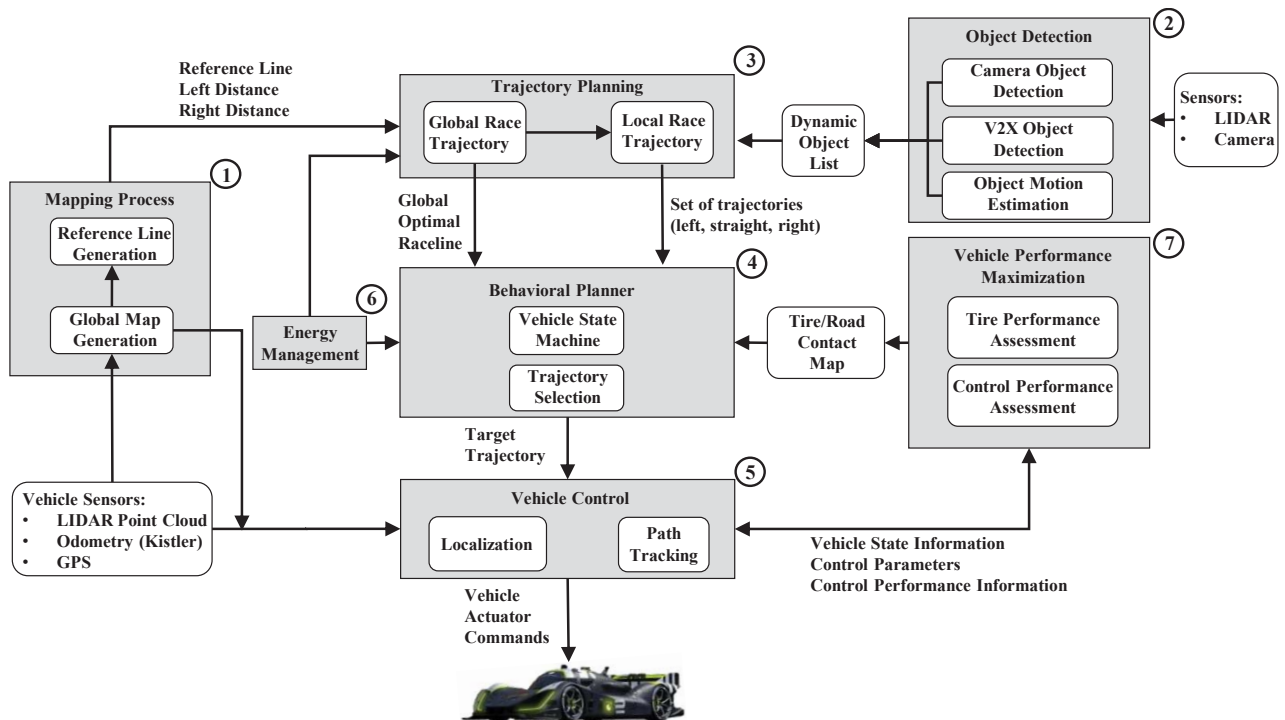


Fig. 2. TUM Roborace software architecture for Roborace Season Alpha 2019

In the last part, a controller makes sure that the planned trajectory (position and velocity) will be followed correctly.

This rather generic approach is extended by Serban et al. [6] who were claiming that previous architectural solutions like [7]–[9] are often an after-math of building or evolving an autonomous vehicle and are not the result of a clear software development lifecycle. Even though they are following the functional requirements from automotive standards, e.g. the incremental development process of autonomous vehicles and software, their architecture is lacking a final validation and testing in a real vehicle.

Having all this in mind, the most relevant work for this paper is the software architecture we were using during the Roborace Berlin event in 2018 presented in [4]. Based on the current state of the art of hardware used in autonomous vehicles (e.g. Nvidia Drive PX2), this software proved its capabilities and robustness by beating the lap time of a human driver. Due to the contest regulations this software architecture missed a few essential functions: First of all, the car did not need to provide an object list for surrounding objects because it was driving alone on the racetrack. Secondly, the car was not able to calculate new trajectories dynamically and therefore no overtaking or evasive maneuvers could be made. Thirdly, the car was not able to really drive at the dynamic handling limits because there did not exist a function to approach them during the race. Because of this and the Roborace contest regulations for 2019, it is our goal to enhance the current architecture and to integrate further functions.

III. SOFTWARE ARCHITECTURE

In the following section we explain in detail the new software architecture we are proposing. In addition, we add the related work for the individual algorithms. The overall software architecture is displayed in figure 2 and is splitted into seven different submodules.

A. Submodule 1: Mapping Process

The Roborace vehicle is designated for driving on race-tracks. Therefore, we propose the creation of a global map of the racetrack in an offline mapping process. A manual driver drives three laps that are used to record LIDAR and GPS data. In a first step, we use a mapping algorithm (e.g. gmapping [10]) for creating a 2D-occupancy grid map of the racetrack [11]. In the second step, we enhance the global map with additional information: GPS reference point, direction of travel around the track, left and right side related to the centerline and track features (e.g. overtaking zones). This global map can now be used in the AMCL algorithm for localization of the vehicle [12]. Furthermore, it is the starting point for all trajectory planning, e.g. to create the global race trajectory or detecting the free space of the circuit [13].

B. Submodule 2: Object Detection

A big difference to our first season is that the new software architecture now has to deal with additional vehicles on the racetrack (opponent race cars). Therefore, it was necessary to integrate a submodule for object detection. This submodule consists of three functions that create a dynamic object list of all the vehicles on the racetrack.

Firstly, the Robocar is equipped with three Sekonix SF3326-10X cameras (front, right and left side). The Nvidia Drive PX2 (one of the central ECUs in the vehicle) provides two Tegra GPUs. Therefore, we decided to use an artificial neural network for object detection and tracking. We trained the network with images from different Robocars, Formula 1 cars and other racing vehicles. Afterwards, we optimized the neural network regarding the real-time application with TensorRT for running on the GPUs. Secondly, a V2X interface is used for providing a minimum set of information, e.g. the position on the track and the velocity, to the other cars and supplying their information to own trajectory and behavior planning. The V2X data is used both, as a ground truth for training the neural network regarding the object tracking and as a fallback solution if the object detection via camera fails. Thirdly, because both object detection and V2X information do not provide the planned trajectory of the opponent vehicles, we need an additional function for estimating their motion (Prediction). The methods for estimating motions can be subdivided into physics-, maneuver- and interaction-based approaches based on their degree of abstraction [14]. For all these approaches a lot of research was already conducted and various methods were proposed and tested. We evaluated them regarding the necessary data, complexity, flexibility, computing power, prediction horizon, consideration of uncertainties, stochastic representation of results and clear traceability. The analysis showed that none of the existing approaches is suitable for direct adaption to the racing scenario. For this reason, a new approach will be developed based on the existing concepts. The plan is to establish a hybrid approach with a physics- and a maneuver-based component. The maneuver-based component is chosen because it allows prediction over a period of several seconds using information about the static environment and the current situation. The physics-based approach is independent of this data and is based exclusively on the current state of a detected vehicle. As a result, meaningful prediction is only possible for a reduced period of time, but unexpected movement can also be taken into account. By combining the approaches in a hybrid system their respective strengths are used.

C. Submodule 3: Trajectory Generation

The trajectory generation is the main part of the software architecture and is split into a global race trajectory planner and a local race trajectory planner.

The global trajectory planner is executed offline before the race. It generates a time optimal trajectory regarding the vehicle physics on the basis of the static map. Most of the related work can only be applied for medium to lateral accelerations and at low velocities. In the context of high performance cars and racecars the amount of literature is much smaller. Several authors [15], [16] propose to use optimal control for this purpose. Another approach is the offline optimization of pre-planned maneuvers [17] which can then be composed online according to the traffic driving situation. Our first algorithm was based on the approach

by Braghin et al. [18]. It uses a geometrical optimization of the path along the race track to minimize the sum of the discretized curvature. For the upcoming season, this was replaced by a lap time minimization using optimal control techniques based on [19], [20]. It creates a global optimal trajectory which is nearly 1.5 s faster than the previous approach (figure 3). Another improvement was the introduction of a spline approximation for the preprocessing of the centerline. It allows us to handle noisy curvature profiles resulting from the grid map discretization and therefore leads to much better optimization results.

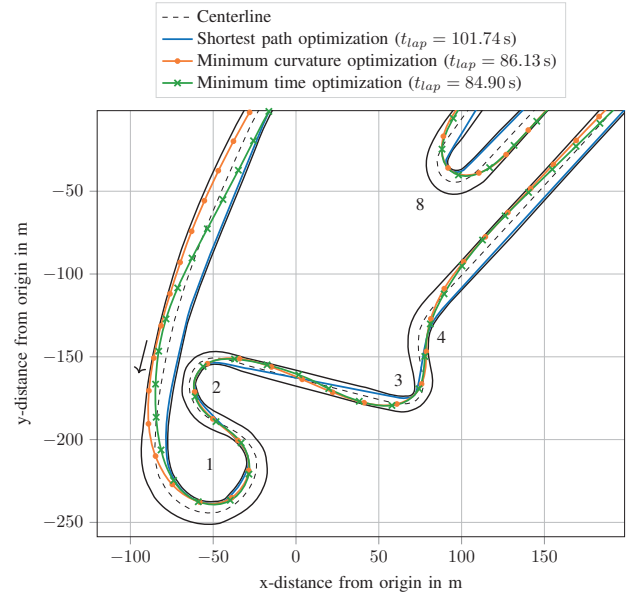


Fig. 3. Optimized race trajectories based on different approaches

The general goal is to follow this time optimal trajectory during the race as long as possible. However, if other cars or static obstacles are around the racecar we have to react to them, e.g. to overtake them. Therefore, we have to plan a local trajectory that is capable of handling different scenarios dynamically. One important aspect is that the vehicle's handling limits have to be considered also during the local planning. Another aspect is that the local planner should always follow the goal to come back to the global race trajectory after an object is passed. This ensures to drive the best possible lap times.

In the past decades, several approaches tackling local trajectory planning have been proposed. Recent surveys [21]–[23] summarize and cluster the existing methods. Thereby – given the problem of local trajectory planning – optimization and graph-based techniques are the most promising solutions. Optimization based motion planning is an active research field with an immense amount of publications, especially dedicated to model predictive control (MPC). A major difficulty lies in the tendency of these methods to detect local minima only. A common approach is to formulate the planning problem in a convex manner, which often restricts the solution space. Most of the research focuses on urban

or highway applications, which differ often with respect to their assumptions, requirements and boundary conditions [21]–[23]. A few research projects [24]–[26] address autonomous racing with optimization methods ranging from optimal control to learning MPC. None of these approaches addresses dynamical objects, which would require longer planning horizons in semistructured environments and would question the real time applicability. Graph-based algorithms have proved to be effective for autonomous driving [27]–[31]. Several authors [27]–[29] span a one layered graph in the spacial dimension, which offers various lateral and sometimes longitudinal target states along the track. Some of the approaches include a dynamic object handling within a short time horizon. Gu et al. [30] extend the spacial dimension to a hierarchical tree with multiple linear edges covering the track region. McNaughton et al. [31] spanned a spatiotemporal hierarchical tree. However, to serve a race scenario, the range of velocities and acceleration profiles needed within a spatiotemporal approach would be too large for a real time graph representation. Besides, one layered graphs lack the capability to model sophisticated maneuvers, like an overtaking maneuver.

We address these issues by spanning a spacial hierarchical tree with a far planning horizon and plan the temporal dimension in an adjacent step. We extend the concept of Gu et al. [30] by splitting the approach in an offline and online phase, sampling of realistic probe paths instead of linear ones and the usage of action sets. The approach is describe in the paper of Stahl et al. [32] more in detail.

1) **Offline:** Within the offline part of the local planner, we generate a state lattice covering the complete track. The lattice is generated on the basis of the normals of the centerline which was created in section III-A. Each normal vector holds a fixed number of nodes, whereby the nodes of adjacent layers are connected by C^1 continuous cubic splines (edges). Afterwards, a cost function is used to calculate a dedicated cost value for every spline segment in the lattice. Thereby, the cost value represents the undesirability of the vehicle following that path. Two major metrics within the cost function penalize the curvature of the spline segment as well as lateral displacement to the race line. All the information derived in the offline phase is stored to a file and reduces the online computation load.

2) **Online:** During online execution, the offline generated graph is further progressed in order to generate an action set. First, a local node template has to be extracted from the full graph. Thereby, the layers ranging from the ego-vehicles position to a desired planning horizon (e.g. 200 m) are extracted from the overall graph (Figure 4, highlighted in green). Second, a subset of the local node template is extracted for each action primitive to be provided to the behavior planner. The goal is to provide three action primitives: “straight”, “left” and “right”. Within the “left” and “right” set, the edges colliding with a potential lead vehicle and it’s prediction are removed from the graph. Furthermore, the nodes to the left or right of the lead vehicle are removed, in order to restrict the solution of the graph search to the respective opposite

side. Third, a shortest path search is triggered in each of the action templates (schematically shown for a “right”-primitive in Figure 4). Action primitives holding no feasible graph-search solution are removed from the set. Finally, each node sequence is post processed by calculating a C^2 continuous splines and generating a velocity profile using the approach proposed by Heilmeyer et al. [33]. It should be noted here, that the velocity profile for the “straight” primitive is altered according to a PD-control law, whenever a lead vehicle is within the planning horizon.

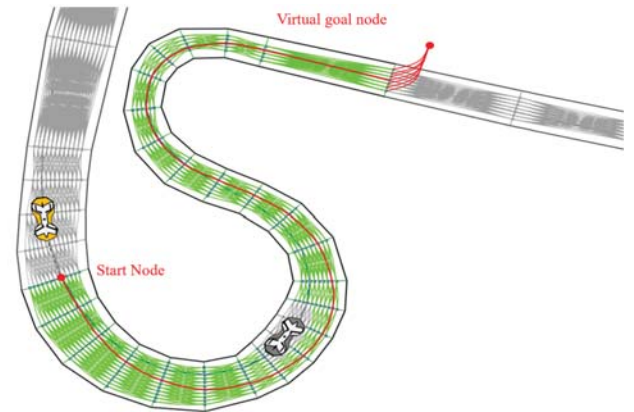


Fig. 4. Dynamic local path planning with the graph based approach

D. Submodule 4: Behavior Planner

The behavior planner module consists of two components: a **vehicle state machine** and a trajectory assessment. The vehicle state machine handles the transition between the various states of the racecar, e.g. standstill, driving and emergency. It is also responsible for the startup process of the vehicle. This means that it launches all required subprocesses. Furthermore, the statemachine monitors all connections to the other submodules using watchdog counters and enters an emergency state if any connection is lost. Lastly, it monitors the driving task, i.e. the number of laps to drive.

The **trajectory assessment functionality** is strongly connected to our local trajectory planner concept. It receives a limited amount of collision-free local trajectories (e.g. 50) that are divided into three action sets: “straight”, “left” and “right”. These trajectories are then further evaluated based on a multi-stage decision process. Assessing them in more detail is necessary to assure a certain safety level, e.g. by taking uncertainties in the pose estimates into account. Due to computation time constraints this cannot be done directly in the local trajectory planner for all possible trajectories.

The general goal of the assessment is to select the fastest local trajectory that fulfills all safety requirements. The multi-stage process is favorable over a multi-objective target function as it guarantees that every chosen trajectory really fulfills these requirements. Furthermore, it reduces the computation time as more and more trajectories are dismissed after every stage of the process. It consists of five stages:

- **Stage 1:** Check if overtaking is allowed and if overtaking trajectories are available
- **Stage 2:** Check if remaining trajectories end on the global race trajectory
- **Stage 3:** Check if remaining trajectories keep safety distance to static objects
- **Stage 4:** Check if remaining trajectories keep safety distance to dynamic objects
- **Stage 5:** Select fastest of the remaining trajectories (requires comparable basis)

Firstly, the function simply checks if overtaking is currently allowed by the race control and if any overtaking trajectory is available. If one of both is not the case, it just returns the best “straight” trajectory. In this case the car just follows the global raceline and stays behind any object in front. Secondly, all local trajectories are dismissed that do not end on the global race trajectory. This is necessary to assure that we are able to finish the overtaking maneuver within the calculation horizon of the local trajectory planner. Otherwise it could happen that we are too fast in the next corner. Thirdly and fourthly, we check if the remaining local trajectories keep a specified safety distance to every object around us. Here we consider uncertainties in the pose estimates of both, our car and the other objects. In the last step we want to select the fastest of all remaining local trajectory. However, they cannot be compared directly because of their different lengths. To obtain a comparable basis, the local trajectory with the greatest progress on the distance of the global raceline is taken as a reference. The shorter local trajectories are evaluated until their end and then completed by the remaining part of the global trajectory. This is possible without problems due to stage two.

The selected local trajectory is then post-processed. This includes interpolation to a velocity dependent step size and calculation of an emergency trajectory containing a modified velocity profile. The result is then sent to the vehicle control.

E. Submodule 5: Vehicle Control

Similar to last year’s architecture we split the control of the vehicle into a trajectory-tracking and low-level control part (figure 5).

We track the trajectory using a PD-like-Controller with gain scheduling by measuring the lateral deviation and the velocity heading error. The low-level velocity controller is a P-controller combined with feedback of a disturbance estimator. It calculates the final force request for the electrical engines of the racecar. What’s new in this season is the low-level curvature controller. We enhanced the P-controller of the curvature tracking by integrating a learning feedforward controller. In this learning algorithm, we constantly update the relation between curvature request, velocity and the resulting measured curvature. The result is stored in form of a corrective steering angle added to the neutral-steer feedforward term (figure 6). This enables to compensate for steering calibration errors and steady-state under- and oversteer.

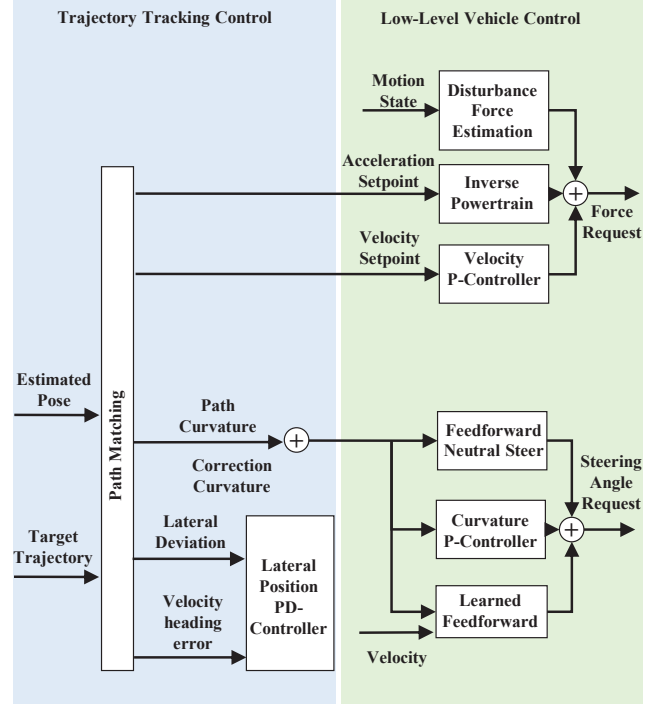


Fig. 5. Control Module setup

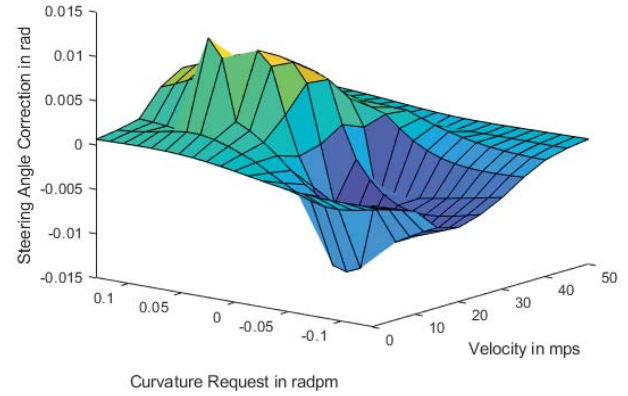


Fig. 6. Learned corrective steering based on vehicle target operating point

Based on the data we gain from driving with the vehicle we use a Gaussian Process regression to identify the necessary correction angle to decrease the lateral control error of the vehicle at high velocities. As a result, we are able to decrease the absolute lateral control error in average by 52 % which can be seen for one lap in figure 7.

The localization of the vehicle is again done with a Kalman Filter that fuses the information from GPS, odometry from IMU and optical sensor and AMCL localization from LIDAR point clouds. A detailed description of the localization algorithms can be found in [34] and [35].

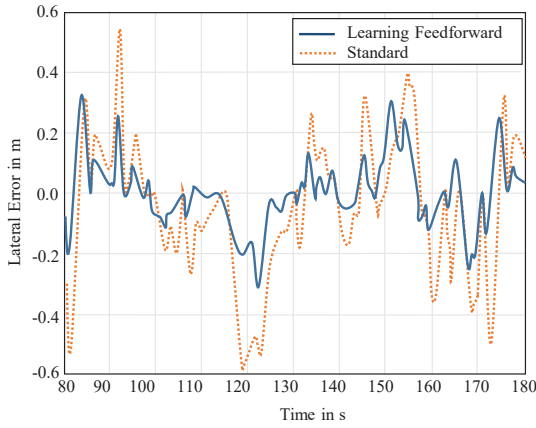


Fig. 7. Comparison of lateral tracking performance for the controller with and without learning feedforward in a sophisticated dual-track model simulation.

F. Submodule 6: Energy Management Strategy

Similar to the Formula E, the Roborace car has a limited battery capacity that is not capable of delivering full power for the full race distance. Therefore, we need an energy management submodule that supervises the electrical drivetrain (e.g. state of charge, temperature, engine power, torque, etc.). In addition to this supervision, we extend the calculation of the globally optimal minimum time trajectory from subsection C by taking a limited energy amount into account. With the help of these trajectories, the behavior planner could react to opponent cars by adapting the race strategy, e.g. by saving some energy at the beginning of the race that can be used as a boost later in the race. Further details are described in [36].

G. Submodule 7: Vehicle Performance Assessment

Last year's architecture was focused on driving fast and safe with the vehicle on a known track. This year's goals are to drive fast and safe again but now with handling the racecar at the physical limits. Therefore, we created an additional submodule which constantly adapts the vehicle to the current track performance. The vehicle performance assessment is split into two parts:

1) *Tire Performance Assessment*: The main goal of the tire performance assessment is to obtain information about the friction potential of the vehicle's tire-road contact and to store and provide this data for use in subsequent laps. With this knowledge, both planning algorithms for global raceline and local trajectory calculate an adjusted path and velocity profile taking the vehicle's local maximum lateral and longitudinal acceleration into account. A central aspect of the algorithm is a high-resolution map which stores, manages and distributes local data to other algorithms relying on that information. By using vehicle-internal state variables (accelerations, forces, torques) and an Extended Kalman Filter for estimating (partially) unobserved state variables, the current tire-road friction potential can be estimated.

Furthermore, the algorithm collects data over a certain period of time to calculate a mean value of the friction potential for fixed track segments. The tire-road friction potential map of the racetrack is split into quadratic segments of 0.5×0.5 m to store the obtained data. This map allows us to adjust the global raceline and the local trajectories in order to maximize friction exploitation (Figure 8).

Future work will focus on a prediction algorithm with the goal of predicting the tire-road friction potential to obtain information about track areas in front of the vehicle or besides the raceline [37].

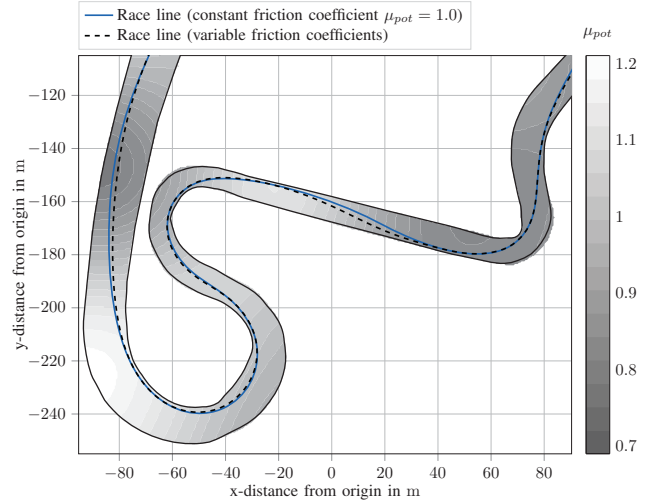


Fig. 8. Two different calculated global racelines based on data from the tire-road friction potential map.

2) *Control Performance Assessment*: In our current software architecture we plan under ideal and simplified conditions. Obviously, this leads to a mismatch between planning and realization of the race trajectory. To mitigate this uncertainty, it is required to include safety margins in terms of maximum accelerations and distance to the track boundaries, however it is not clear a priori how to set them as they depend on the controller performance and the environment conditions. To overcome this problem, a data-driven algorithm is used which scales the allowed maximum accelerations to minimize lap-time while at the same time monitoring several safety constraints (figure 9). Its mathematical foundations are based upon the safe optimization algorithm presented in [38].

As safety constraints we choose the peak lateral tracking error, the longitudinal wheelslip and the difference between front and rear sideslip angle. By using a Gaussian Process regression, we learn those constraints from using real data and can improve the estimates after each lap. The uncertainty calculated by the Gaussian Process framework is used to control the exploration process. Sample points which are far away from already available data have a higher uncertainty which leads to cautious exploration in those areas.

IV. CONCLUSION AND DISCUSSION

In this paper, we presented the next software release of the TUM Roborace team by displaying the new holistic software

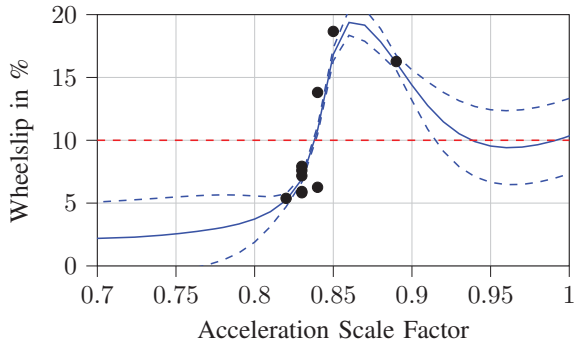


Fig. 9. Learning result for the longitudinal wheelslip maximum used within the control performance assessment algorithm. The red line depicts the maximum allowed wheelslip for a scale factor to be classified safe.

structure for autonomous driving based on the hardware used in the Roborace racecar. This software is implemented on the Roborace Devbot 2.0 and tested in the Roborace Season Alpha 2019. In comparison to the software we were using in the Roborace Berlin Event 2018 the software is improved in several ways. We integrated a real time vehicle detection algorithm and complemented the global trajectory planner by a local trajectory planner to allow overtaking and evasion maneuvers taking static and dynamic objects on the racetrack into account. The decision for this is made in the behavior planner. Furthermore, we developed software for a continuous improvement of the vehicle performance in the race. Tire-road friction potential estimating algorithms increases the utilization of the lateral and longitudinal acceleration potentials. A safe learning control algorithm finds the best scale factors for lateral and longitudinal accelerations.

This paper presents the overall structure and is giving an idea about the different software parts and how they fit together. Each software part, e.g. localization, trajectory generation and control of the vehicle, is too complex for explaining it in one paper in detail. The authors therefore aim for publishing each software part in an individual paper. For further interest we want to point out that the software parts are published under a open-source license on Github [39].

CONTRIBUTIONS

Johannes Betz as the first author initiated the idea of this paper and contributed essentially to the overall structure of the software and this paper. Alexander Wischnewski (vehicle control, control performance assessment), Alexander Heilmeyer (global trajectory planning, behavior planning), Felix Nobis (mapping process, object detection), Tim Stahl (local trajectory planning), Leonhard Hermansdorfer (tire performance assessment) and Thomas Herrmann (energy management strategy) contributed equally to different parts of the software architecture and of this paper. Markus Lienkamp made an essential contribution to the conception of the research project. He revised the paper critically for important intellectual content. He gave final approval of the version to be published and agrees to all aspects of the work.

As a guarantor, he accepts the responsibility for the overall integrity of the paper.

ACKNOWLEDGEMENT

First, we want to thank the Roborace team for giving us the opportunity to work with them and using their Devbot vehicles for our research. We would like to thank the Bayerische Forschungsstiftung for funding us in the research project rAIcing. We would like to thank Continental Engineering Services and the TÜV Süd for funding us with research projects. The work described in this paper was also supported by the basic research fund of the Chair of Automotive Technology from the Technical University of Munich. In addition, we want to thank in-tech for supporting us with the Speedgoat software development.

REFERENCES

- [1] Verband der Automobilindustrie, "Automatisiertes Fahren," 2019, accessed on the 25.05.2019. [Online]. Available: <https://www.vda.de/de/themen/innovation-und-technik/automatisiertes-fahren/automatisiertes-fahren.html>
- [2] Roborace Ltd. (2017, accessed on the 25.05.2019) Roborace. [Online]. Available: <https://roborace.com/>
- [3] J. Betz, A. Wischnewski, A. Heilmeyer, F. Nobis, T. Stahl, L. Hermansdorfer, and M. Lienkamp, "What can we learn from autonomous level-5 motorsport?" in *Proceedings*. Springer Fachmedien Wiesbaden, sep 2018. [Online]. Available: https://doi.org/10.1007/978-3-658-22050-1_12
- [4] —, "A software architecture for an autonomous racecar," in *2019 IEEE 89th Vehicular Technology Conference (VTC2019-Spring)*. IEEE, Apr. 2019. [Online]. Available: <https://doi.org/10.1109/vtcspring.2019.8746367>
- [5] S. Pendleton, H. Andersen, X. Du, X. Shen, M. Meghjani, Y. Eng, D. Rus, and M. Ang, "Perception, planning, control, and coordination for autonomous vehicles," *Machines*, vol. 5, no. 1, p. 6, feb 2017. [Online]. Available: <https://doi.org/10.3390/machines5010006>
- [6] A. C. Serban, E. Poll, and J. Visser, "A standard driven software architecture for fully autonomous vehicles," in *2018 IEEE International Conference on Software Architecture Companion (ICSA-C)*. IEEE, apr 2018. [Online]. Available: <https://doi.org/10.1109/icsa-c.2018.00040>
- [7] S. Behere and M. Törngren, "A functional reference architecture for autonomous driving," *Information and Software Technology*, vol. 73, pp. 136–150, may 2016. [Online]. Available: <https://doi.org/10.1016/j.infsof.2015.12.008>
- [8] M. Montemerlo, J. Becker, S. Bhat, H. Dahlkamp, D. Dolgov, S. Ettinger, D. Haehnel, T. Hilden, G. Hoffmann, B. Huhne, D. Johnston, S. Klumpp, D. Langer, A. Levandowski, J. Levinson, J. Marcil, D. Orenstein, J. Paefgen, I. Penny, A. Petrovskaya, M. Pflueger, G. Stanek, D. Stavens, A. Vogt, and S. Thrun, "Junior: The stanford entry in the urban challenge," *Journal of Field Robotics*, vol. 25, no. 9, pp. 569–597, sep 2008. [Online]. Available: <https://doi.org/10.1002/rob.20258>
- [9] K. Jo, J. Kim, D. Kim, C. Jang, and M. Sunwoo, "Development of autonomous car—part i: Distributed system architecture and development process," *IEEE Transactions on Industrial Electronics*, vol. 61, no. 12, pp. 7131–7140, dec 2014. [Online]. Available: <https://doi.org/10.1109/tie.2014.2321342>
- [10] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with rao-blackwellized particle filters," *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34–46, Feb. 2007. [Online]. Available: <https://doi.org/10.1109/tro.2006.889486>
- [11] F. Nobis, J. Betz, L. Hermansdorfer, and M. Lienkamp, "Autonomous racing: A comparison of slam algorithms for large scale outdoor environments," in *Proceedings of the 2019 3rd International Conference on Virtual and Augmented Reality Simulations - (ICVARS 19)*. ACM Press, 2019. [Online]. Available: <https://doi.org/10.1145/3332305.3332319>

- [12] D. Fox, W. Burgard, F. Dellaert, and S. Thrun, "Monte carlo localization: Efficient position estimation for mobile robots," in *IN PROC. OF THE NATIONAL CONFERENCE ON ARTIFICIAL INTELLIGENCE (AAAI)*, 1999, pp. 343–349.
- [13] P. R. Palafox, J. Betz, F. Nobis, K. Riedl, and M. Lienkamp, "SemanticDepth: Fusing semantic segmentation and monocular depth estimation for enabling autonomous driving in roads without lane lines," *Sensors*, vol. 19, no. 14, p. 3224, July 2019. [Online]. Available: <https://doi.org/10.3390/s19143224>
- [14] S. Lefèvre, D. Vasquez, and C. Laugier, "A survey on motion prediction and risk assessment for intelligent vehicles," *ROBOMECH Journal*, vol. 1, no. 1, jul 2014. [Online]. Available: <https://doi.org/10.1186/s40648-014-0001-z>
- [15] L. Alexander, D. Alexander, and M. Manfred, "Optimization-based autonomous racing of 1:43 scale rc cars," *Optimal Control Applications and Methods*, vol. 36, no. 5, pp. 628–647, 2014.
- [16] A. Katriniok and D. Abel, "Ltv-mpc approach for lateral vehicle guidance by front steering at the limits of vehicle dynamics," in *2011 50th IEEE Conference on Decision and Control and European Control Conference*, Dec 2011, pp. 6828–6833.
- [17] S. Glaser, B. Vanholme, S. Mammar, D. Gruyer, and L. Nouveliere, "Maneuver-based trajectory planning for highly autonomous vehicles on real road with traffic and driver interaction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 11, no. 3, pp. 589–606, Sept 2010.
- [18] F. Braghin, F. Cheli, S. Melzi, and E. Sabbioni, "Race driver model," *Computers and Structures*, vol. 86, no. 13–14, pp. 1503–1516, July 2008.
- [19] G. Perantoni and D. J. Limebeer, "Optimal control for a formula one car with variable parameters," *Vehicle System Dynamics*, vol. 52, no. 5, pp. 653–678, 2014.
- [20] I. Gundlach, U. Konigorski, and J. Hoedt, "Zeitoptimale Trajektorienplanung für automatisiertes Fahren im fahrdynamischen Grenzbereich," *VDI-Berichte*, no. 2292, pp. 223–234, 2017.
- [21] D. Gonzalez, J. Perez, V. Milanés, and F. Nashashibi, "A review of motion planning techniques for automated vehicles," *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 4, pp. 1135–1145, 2016.
- [22] B. Paden, M. Cap, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *IEEE Transactions on Intelligent Vehicles*, vol. 1, no. 1, pp. 33–55, 2016.
- [23] C. Katrakazas, M. Qudus, W.-H. Chen, and L. Deka, "Real-time motion planning methods for autonomous on-road driving: State-of-the-art and future research directions," *Transportation Research Part C: Emerging Technologies*, vol. 60, pp. 416–442, 2015.
- [24] G. Williams, P. Drews, B. Goldfain, J. M. Reh, and E. A. Theodorou, "Aggressive driving with model predictive path integral control," in *2016 IEEE International Conference on Robotics and Automation, Stockholm, Sweden, May 16th–21st*, A. Okamura and A. Menciassi, Eds. Piscataway, NJ: IEEE, 2016, pp. 1433–1440.
- [25] T. Rizano, D. Fontanelli, L. Palopoli, L. Pallottino, and P. Salaris, "Local motion planning for robotic race cars," in *2013 IEEE Annual Conference on Decision and Control*. Piscataway, NJ: IEEE, 2013.
- [26] M. Brunner, U. Rosolia, J. Gonzales, and F. Borrelli, "Repetitive learning model predictive control: An autonomous racing example," in *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*. Piscataway, NJ: IEEE, 2017, pp. 2545–2550.
- [27] X. Li, Z. Sun, D. Cao, D. Liu, and H. He, "Development of a new integrated local trajectory planning and tracking control framework for autonomous ground vehicles," *Mechanical Systems and Signal Processing*, vol. 87, pp. 118–137, 2017.
- [28] X. Hu, L. Chen, B. Tang, D. Cao, and H. He, "Dynamic path planning for autonomous driving on various roads with avoidance of static and moving obstacles," *Mechanical Systems and Signal Processing*, vol. 100, pp. 482–500, 2018.
- [29] M. Werling, J. Ziegler, S. Kammel, and S. Thrun, "Optimal trajectory generation for dynamic street scenarios in a frenét frame," in *IEEE International Conference on Robotics and Automation*, 2010, pp. 987–993.
- [30] T. Gu, J. Snider, J. M. Dolan, and J.-w. Lee, "Focused trajectory planning for autonomous on-road driving," in *2013 IEEE Intelligent Vehicles Symposium*. IEEE, 2013, pp. 547–552.
- [31] M. McNaughton, C. Urmson, J. M. Dolan, and J.-w. Lee, "Motion planning for autonomous driving with a conformal spatiotemporal lattice," in *2011 IEEE International Conference on Robotics and Automation*, A. Bicchi, Ed. Piscataway, NJ: IEEE, 2011, pp. 4889–4895.
- [32] T. Stahl, W. Alexander, J. Betz, and M. Lienkamp, "Multilayer graph-based trajectory planning for race vehicles in dynamic scenarios," in *2019 IEEE 22nd International Conference on Intelligent Transportation Systems (ITSC 2019)*. IEEE, 2019, submitted and accepted.
- [33] A. Heilmeyer, A. Wischniewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann, "Minimum curvature trajectory planning and control for an autonomous race car," *Vehicle System Dynamics*, pp. 1–31, June 2019. [Online]. Available: <https://www.tandfonline.com/doi/full/10.1080/00423114.2019.1631455>
- [34] T. Stahl, A. Wischniewski, J. Betz, and M. Lienkamp, "ROS-based localization of a race vehicle at high-speed using LIDAR," *E3S Web of Conferences*, vol. 95, p. 04002, 2019. [Online]. Available: <https://doi.org/10.1051/e3sconf/20199504002>
- [35] A. Wischniewski, T. Stahl, J. Betz, and M. Lienkamp, "Vehicle dynamics state estimation and localization for high performance race cars," submitted to: *Proceedings of 10th IFAC Symposium on Intelligent Autonomous Vehicles*, 2019, in Press.
- [36] L. Hermansdorfer, J. Betz, and M. Lienkamp, "A concept for estimation and prediction of the tire-road friction potential for an autonomous racecar," in *2019 IEEE 22nd International Conference on Intelligent Transportation Systems (ITSC 2019)*. IEEE, 2019, submitted and accepted.
- [37] T. Herrmann, F. Christ, J. Betz, and M. Lienkamp, "Energy management strategy for an autonomous electric racecar using optimal control," in *2019 IEEE 22nd International Conference on Intelligent Transportation Systems (ITSC 2019)*. IEEE, 2019, submitted and accepted.
- [38] F. Berkenkamp, A. P. Schoellig, and A. Krause, "Safe controller optimization for quadrotors with Gaussian processes," in *Proc. of the IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 493–496. [Online]. Available: <https://arxiv.org/abs/1509.01066>{arXiv:1509.01066[cs.RO]}
- [39] Technical University of Munich - Chair of Automotive Technology. (2019, accessed on the 25.05.2019) TUM Roborace Team Software. [Online]. Available: https://github.com/TUMFTM/veh_passenger